AI-POWERED HIRING DECISION SUPPORT

Smart Recruitment Screening Assistant

Using candidate demographics, training history, and employment signals to flag which applicants are likely to disengage — so recruiters can prioritize candidates most likely to advance and stay.

Final Project Presentation

INDUSTRY CONTEXT

Every vacancy draws hundreds of applications

01

The volume problem

Recruiting teams can't manually review every applicant profile at the scale modern hiring requires — screening needs to scale with volume.

02

The signal

This dataset's "job-seeking intent" label doubles as an engagement / flight-risk signal — a proxy for how likely a candidate is to stay committed through the process.

03

The ask

Predict, from data available at application, which candidates are safe to advance and which should be flagged for closer review before the next hiring stage.

Goal: a binary screening classifier — Recommend to Advance vs Flag for Review

THE DATASET

19,158 candidates, 14 attributes, one target

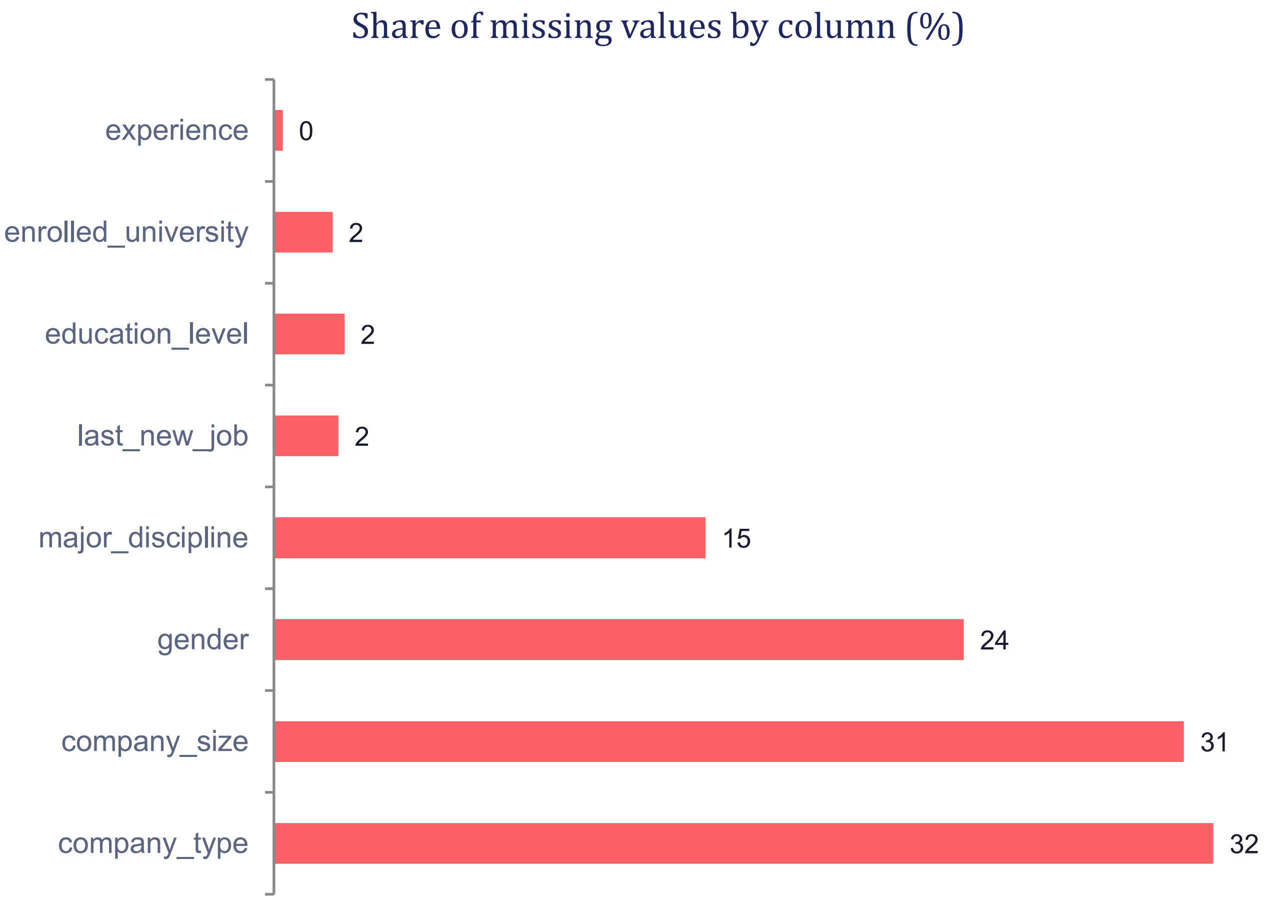
19,158	13	24.9%	97
candidate records	predictor features	flagged as flight-risk	duplicate rows removed

Feature groups

- Demographic — gender, city, city development index
- Education — enrolled_university, education_level, major_discipline
- Career history — relevent_experience, experience, last_new_job
- Employer — company_size, company_type
- Engagement — training_hours

DATA QUALITY

Missing values were concentrated in a few columns



Approach: mode imputation

- Every categorical column with missing values was filled with its most frequent category (mode).

Kept every row instead of dropping ~32% of the data on company_type / company_size alone.

Simple, transparent, and consistent across all 8 affected columns.

PREPARATION

From raw survey fields to model-ready features

1

Clean

Dropped 97 duplicate rows and the non-predictive enrollee_id column.

2

Recode

experience: "<1"→0, ">20"→21. last_new_job: "never"→0, ">4"→5. Both cast to integers.

3

Encode

Label-encoded the target; one-hot encoded 8 categorical columns (city, gender, education, etc.).

4

Scale

80/20 stratified train-test split, then StandardScaler fit on training data only.

BUSINESS INTELLIGENCE

Who is actually in the applicant pool

CANDIDATE PROFILE TRENDS

Education level

Graduate 63% · Masters 23% · High School 10% · Phd/Primary 4%

Relevant experience

Has relevant experience 72% · No relevant experience 28%

University enrollment

Not enrolled 74% · Full-time course 20% · Part-time course 6%

Major discipline

STEM 90% · Non-STEM / Other 10%

RECRUITMENT STATISTICS

Current employer type

Private Ltd 83% · Funded Startup 5% · Public Sector 5% · Early-stage/NGO/Other 7%

Gender

Male 93% · Female 6% · Other 1%

Years since last job change

1 year 44% · 2-4 years 26% · 5+ years 17% · Never had one 13%

19,061 applications after de-duplication, spanning 123 distinct cities

MODELING APPROACH

Four classifiers, one class-imbalance strategy

1

Logistic Regression

class_weight = balanced

2

Random Forest

200 trees · class_weight = balanced

3

Gradient Boosting

scikit-learn default depth-3 trees

4

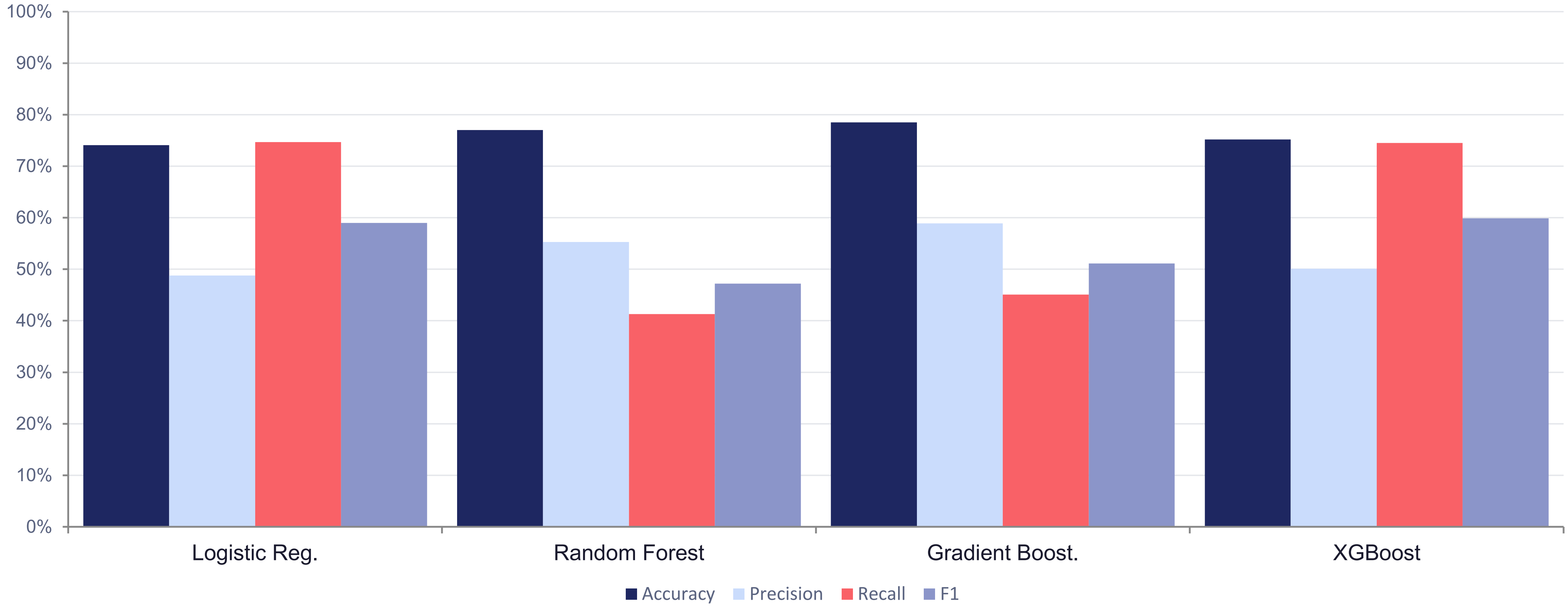
XGBoost

scale_pos_weight = neg/pos ratio

Why weight the classes? Only ~25% of candidates carry a flight-risk signal — an unweighted model can reach ~75% accuracy by recommending "advance" every time, while missing nearly every candidate who needs a closer look.

RESULTS

All four models land in the 74–78% accuracy range



RESULTS

Where each model's screening calls go right — and wrong

Logistic Regression

	Pred. Advance	Pred. Flag
Actual Advance	2128	749
Actual Flag	242	713

Random Forest

	Pred. Advance	Pred. Flag
Actual Advance	2558	319
Actual Flag	561	394

Gradient Boosting

	Pred. Advance	Pred. Flag
Actual Advance	2576	301
Actual Flag	524	431

XGBoost

	Pred. Advance	Pred. Flag
Actual Advance	2170	707
Actual Flag	244	711

Correct call   Screening error

Accuracy alone hides the real trade-off: recall

Gradient Boosting — most precise flags

- Highest accuracy (78.5%) and highest F1 (0.511).

But recall is only 45% — it misses more than half the candidates who are genuinely at flight risk, letting them advance unflagged.

Best fit when flagging a strong candidate for extra review (a false alarm) is the costlier mistake.

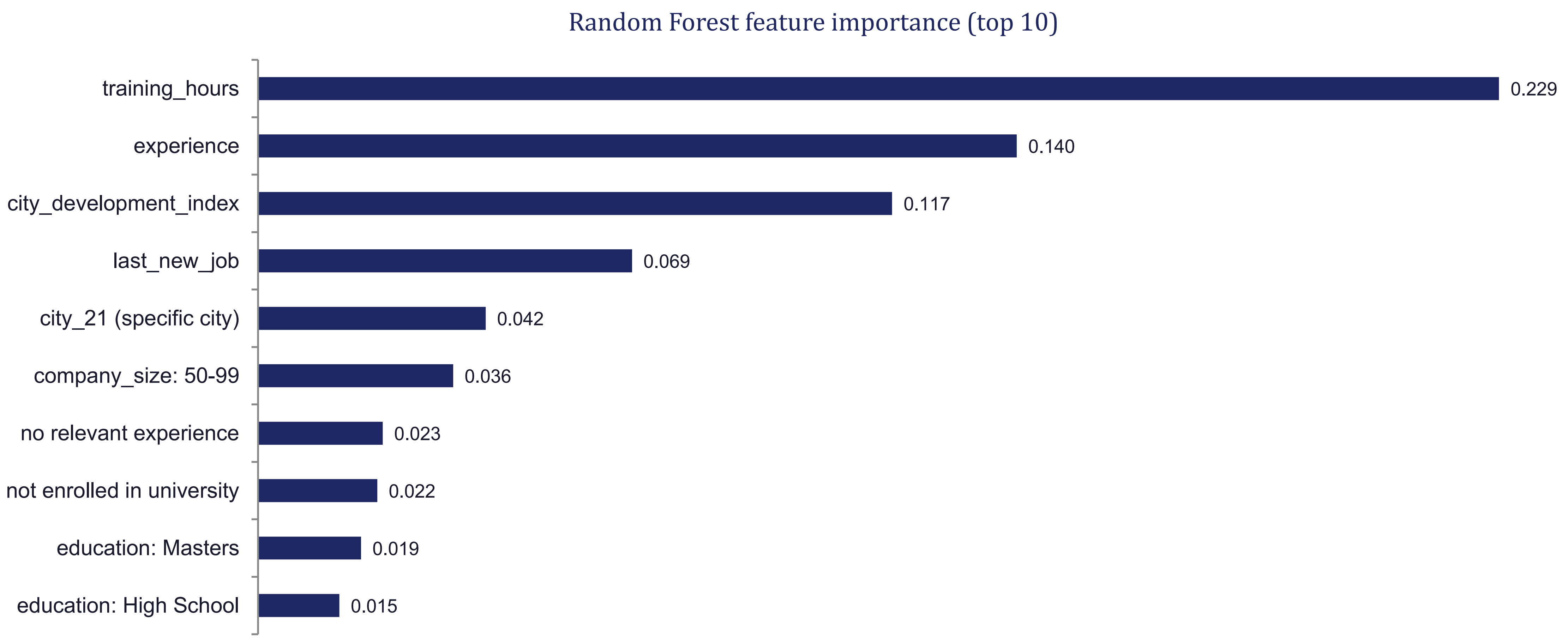
XGBoost / Logistic Reg. — best recall

- Both catch ~75% of genuinely at-risk candidates — nearly double Gradient Boosting's recall.

Trade-off: more false positives, i.e. more strong candidates flagged for review who didn't need to be.

Best fit when letting a flight-risk candidate slip through unflagged is the costlier mistake.

Four features carry most of the signal



10 features capture nearly all the predictive power

Model	Full feature set (~150)	Top 10 features only	Change
Logistic Regression	74.1%	73.1%	-1.0 pt
Random Forest	77.0%	76.2%	-0.8 pt
XGBoost	75.2%	74.8%	-0.4 pt

Takeaway: a lean, 10-feature model is nearly as accurate as the full model — faster to compute, easier to explain to HR stakeholders, and simpler to maintain.

~99%

of full-model accuracy retained, using just 10 of ~150 encoded features

KEY INSIGHTS

What the data says about who to advance

Training investment cuts both ways

training_hours is the single strongest predictor of flight risk — the more training a candidate logs, the more marketable (and mobile) they become.

Experience and tenure matter

Years of experience and time since their last job change are strong signals; both very new and very tenured profiles behave differently.

Local job market shapes risk

city_development_index — a proxy for how developed a candidate's city is — is a top-4 predictor of flight-risk signals.

Weak ties raise the flag

Candidates with no relevant experience or who are not currently enrolled in university show higher odds of being flagged for review.

RECOMMENDATIONS

How recruiters can act on these predictions

1. Screen applicants with the lean 10-feature model for fast, explainable, real-time scoring at the application stage.

2. Use XGBoost or Logistic Regression when the priority is catching as many flight-risk candidates as possible, even with more false alarms.

3. Use Gradient Boosting when a review step is costly and precision matters more than catching every risk case.

4. Route candidates with high training_hours, limited relevant experience, and low city_development_index to a closer review step, not automatic rejection — the model supports the recruiter's decision, it doesn't replace it.

NEXT STEPS

Where this project goes from here

01 Tune the decision threshold to balance recall and precision for the chosen business use case.

02 Run hyperparameter search (grid/Bayesian) on Gradient Boosting and XGBoost for incremental gains.

03 Validate on a hold-out cohort from a later enrollment period to check for drift.

04 Package the 10-feature model behind a simple scoring API for HR dashboards.

Thank you

Questions and discussion welcome.